

Rapport d'audit — Plateforme CleanUx

Audit complet multi-agents (10 dimensions) avec vérification adversariale — 2026-06-08 35 agents, ~2,25M tokens, 50 findings retenus (3 réfutés en vérification).

1. Synthèse exécutive

CleanUx est une plateforme dense et fonctionnellement très avancée, mais **pas prête pour la production en l'état**. L'audit confirme un défaut **critique d'argent réel** : le cron quotidien de payout re-transfère via Stripe les missions déjà payées par destination charge, ouvrant un double-paiement systématique aux prestataires. Trois familles de risques majeurs dominant : (a) l'argent et la fiabilité Stripe (double virement manuel sans idempotence, webhooks Connect perdus, double déduction de commission au ledger), (b) la conformité RGPD (le droit à l'oubli automatique ne s'exécute jamais, anonymisation incomplète laissant survivre adresses et emails), et (c) le cœur métier mobile cassé (le flow QR start/end client tape des endpoints inexistants). À cela s'ajoute une dette structurelle nette : coexistence v2/legacy non finalisée (annulation, présence, onboarding) atteignable simultanément avec des logiques d'argent divergentes, et un système de feature flags dont les kill-switches admin sont des no-ops. Le socle sécurité (isolation par utilisateur sur le cœur du flux, secrets, idempotence des webhooks entrants) et plusieurs suites de tests money sont en revanche solides. **Verdict : corriger les findings critical/high — en priorité la couche argent et RGPD — avant tout go-live.**

2. Tableau de bord

Par sévérité

Sévérité	Nombre
Critical	1
High	11
Medium	25
Low	12
Total	49

Par dimension

Dimension	Critical	High	Medium	Low	Total
money:payments-stripe	1	0	2	0	3
security:authz	0	1	2	1	4
security:injection-secrets	0	0	1	2	3
reliability:queues-webhooks	0	2	2	1	5
compliance:rgpd-pii	0	2	4	1	7
mobile:rn-expo	0	2	3	1	6
archi:deadcode-duplication	0	2	2	1	5
data:integrity-schema	0	0	3	2	5
perf:n+1-indexes	0	1	2	3	6
tests:coverage-gaps	0	1	4	0	5
Total	1	11	25	12	49

3. Findings critiques & high

Thème A — Argent & paiements Stripe

A1. [CRITICAL] Double payout Stripe : le cron re-transfère les missions déjà payées par destination charge

`app/Console/Commands/ProcessProviderPayouts.php:161-219` (Phase 2) + `app/Console/Kernel.php:51` + `app/Services/Missions/MissionLifecycleService.php:263-272`

- **Impact** : double paiement systématique au prestataire, perte d'argent irrécupérable, à chaque run quotidien 02:00.
- **Preuve** : `MissionPaymentService::authorize()` crée le `PaymentIntent` avec `transfer_data.destination` (destination charge) → la part prestataire est transférée automatiquement à la capture. `completeMission()` pose `payout_status='processed'` mais **jamais** `stripe_transfer_id`. `handleTransferCreated` ne renseigne `stripe_transfer_id` que si `transfer.metadata.booking_id` existe — absent sur les transferts auto. La Phase 2 sélectionne `where('payout_status', 'processed')→whereNull('stripe_transfer_id')` puis fait un `Transfer::create()` → second paiement. Le commentaire du code reconnaît lui-même le risque (« must NOT be transferred again »).
- **Correctif** : marquer explicitement les missions auto-transférées (`payout_status='auto_transferred'`) à la capture et les exclure de la Phase 2 ; ne pas se fier à `stripe_transfer_id` (jamais peuplé pour les transferts auto). Idéalement supprimer la Phase 2 pour le modèle destination-charge.

A2. [HIGH] Double virement Stripe sur payout manuel (Transfer::create sans clé d'idempotence) — *dimension reliability*

`app/Console/Commands/ProcessProviderPayouts.php:206`

- **Impact** : un crash entre le `Transfer` réussi côté Stripe et l'UPDATE DB → re-sélection du booking au run suivant → second virement réel.
- **Preuve** : `Transfer::create(['amount'⇒$payoutCents, 'destination'⇒$connectId, ... ])` sans 2e argument `options`. Le marquage `stripe_transfer_id / payout_status='transferred'` n'est écrit qu'après le retour réseau (l.216-219). Sélection vulnérable : `where('payout_status', 'processed')→whereNull('stripe_transfer_id')`.
- **Correctif** : `Transfer::create([ ... ], ['idempotency_key' ⇒ 'payout:booking:'.$booking⇒id])`.

Thème B — Fiabilité & machine à états

B1. [HIGH] Aucune transition d'état : un booking complété peut être ré-annulé ET remboursé

`app/Services/CancellationV2/CancellationEngine.php:108`

- **Impact** : un booking `completed/termine` peut être annulé → calcul d'un refund + remboursement Stripe réel + reversal loyalty/promo. Risque financier direct.
- **Preuve** : `execute()` lit `$statusBefore` (l.128) uniquement pour l'audit, jamais comme garde ; on passe directement à la transaction qui pose `status='annule'`. `quote()` ne lève une exception que si le booking est introuvable. Aucun garde-fou de transition centralisé dans le codebase (`grep canTransition/assertTransition` = uniquement `DisputesCenter`).
- **Correctif** : rejeter (`ValidationException`) si `$statusBefore ∈ completedAliases()/cancelledAliases()` ; introduire un garde unique `BookingStatus::canCancel()` réutilisé par `CancellationEngine` ET `CancelBookingService` legacy.

Thème C — Sécurité / autorisation

C1. [HIGH] IDOR complet sur le module Quality/Inspection (lecture ET écriture, client + provider)

`app/Http/Controllers/Api/Client/QualityInspectionClientController.php:21` ;
`app/Http/Controllers/Api/Provider/QualityInspectionController.php:22-122` ;
`app/Services/Quality/QualityInspectionService.php:210-278`

- **Impact** : n'importe quel utilisateur authentifié peut, par énumération d'ID, (1) lire toute inspection d'autrui (photos avant/après, signatures) ; (2) **altérer** des enregistrements qualité d'autrui — `start`, `submitItem`, `uploadPhoto`, `submit`, et côté client `validate_` (signer/valider au nom du client, créant une `ClientSignature` eIDAS-lite et faisant passer le statut à `validated_client`) et `dispute`. La validation client jalonne la clôture qualité et la libération du paiement → atteinte à l'intégrité, pas seulement fuite.
- **Preuve** : `show()` (l.21) charge et renvoie l'inspection sans aucun `abort/authorize` ; `validate_ / dispute` appellent le service sans vérifier que `$inspection→mission` appartient à `$request→user()`. Côté service, `validateByClient` (l.210) et `dispute` (l.256) ne contrôlent que le statut. Aggravé par M1.
- **Correctif** : `MissionQualityInspectionPolicy` + `$this→authorize()` dans chaque méthode ; vérifier `inspection→mission→booking.customer_user_id == user→id` (client) et `lead_provider_user_id / assignment accepté (provider)` ; dupliquer le garde dans le service (defense-in-depth).

Thème D — Conformité RGPD

D1. [HIGH] Le cron d'effacement appelle une commande inexistante — le droit à l'oubli ne s'exécute JAMAIS automatiquement

`app/Console/Kernel.php:35`

- **Impact** : violation directe et silencieuse de l'art.17. Les utilisateurs ayant exercé leur droit à l'oubli restent en base avec toutes leurs PII, alors que `deletion_scheduled_at` leur a confirmé une suppression.
- **Preuve** : Kernel planifie `gdpr:execute-erasure-requests` mais la commande enregistrée à la signature `gdpr:execute-erasures` (`ExecuteErasureRequestsCommand.php:11`). Le job quotidien échoue avec « Command not found ».

- **Correctif** : aligner le nom dans Kernel.php ; ajouter un test qui affirme que toutes les commandes schedulées sont résolubles (`Artisan::all()` contient chaque signature planifiée).

D2. [HIGH] Anonymisation incomplète : adresses clients, emails d'audit et autres PII survivent à l'effacement

`app/Services/Gdpr/DataErasureService.php:115`

- **Impact** : après un « oubli », l'adresse domicile du client (`bookings.adresse/ville/code_postal`), les textes libres (`feedback.commentaire` , `complaint_cases.subject/description`), `kyc_verifications.metadata` , `notifications.data` et `analytics_events` restent en clair, reliés via FK à l'utilisateur. Le docstring prétend pourtant que les bookings sont anonymisés — ce remplacement n'a jamais lieu.
- **Preuve** : `anonymizeUser/anonymizeV2Modules` (l.120-227) ne touchent que users/profiles + quelques tables v2 ; aucune écriture sur `bookings` / `feedback` / `complaint_cases` / `notifications` . La présence de ces PII est confirmée par `DataExportService.php:135` (`bookings: adresse/ville/code_postal`) et :177 (`complaint_cases`).
- **Correctif** : étendre `anonymizeUser()` à ces tables (pseudonymisation là où la conservation comptable l'exige) + test qui affirme qu'aucune PII réelle ne subsiste après `execute()` .

Thème E — Cœur métier mobile (parité cassée)

E1. [HIGH] Flow QR start/end client cassé — endpoints backend inexistant `mobile/client/src/screens/QRScanScreen.tsx:24-29`

- **Impact** : le cœur du flow (QR start → QR end → capture Stripe) est non fonctionnel sur l'app cliente ; chaque scan retourne 404 (« QR code invalide ou expiré »).
- **Preuve** : POST vers `/client/bookings/{id}/qr-start` et `/qr-end` . Ces routes n'existent pas (`routes/api/client.php` n'a que `show/cancel/eta/rating/tip/tracking/favorite/insurance/payment-intent` ; seul `routes/missions.php:28` a une route **web** employé). L'écran est pleinement câblé (RootNavigator:76, BookingDetailScreen:115-133).
- **Correctif** : ajouter `ClientBookingController::qrStart/qrEnd` (validation du code + lifecycle + capture du PaymentIntent), ou pointer le scan vers les vraies routes ; test e2e que QRScanScreen tape une route enregistrée.

E2. [HIGH] Code de fin provider saisi mais jamais transmis au serveur `mobile/provider/src/screens/MissionExecutionScreen.tsx:83`

- **Impact** : la complétion de mission se fait sans vérification du code de fin, contredisant l'UI et cassant la garantie de fin par code.
- **Preuve** : l'écran collecte `endcode` (input l.164-176, affiché dans la confirmation l.77) mais `handleComplete` appelle `lifecycle.mutate('complete')` sans argument, et `useMissionLifecycle` (provider/src/missions/hooks.ts:48-57) poste un body vide vers `/provider/missions/{id}/{action}` .
- **Correctif** : `lifecycle.mutate({action: 'complete', code:endcode})` → poster `{qr_code:code}` avec validation serveur ; bloquer le bouton si `endcode` requis et vide.

Thème F — Dette architecturale v2/legacy

F1. [HIGH] Deux systèmes d'annulation actifs simultanément avec logique de frais/refund divergente selon le canal

`routes/api/client.php:181`

- **Impact** : pour une même annulation d'un même booking, le montant de frais et le refund Stripe diffèrent selon le canal (web=v2 / mobile=legacy / route V2) — divergence directe sur l'argent, non testée de façon croisée.
- **Preuve** : legacy `CancelBookingService` routé via `POST /api/client/bookings/{booking}/cancel-with-fee` (CancellationController:60) ; V2 `CancellationEngine` via web (MesRendezVousClient:298) ET `POST /api/v2/client/bookings/{booking}/cancel` (v2-shared:96, sans middleware feature). Idem côté provider.
- **Correctif** : une source de vérité unique (router tous les canaux vers `CancellationV2\CancellationEngine` , déprécier le legacy), ou gate par feature flag garantissant le même chemin partout.

F2. [HIGH] Présence v2 et Phase 11 désynchronisées : passer online via presence-v2 ne fait recevoir aucune mission ASAP

`app/Services/Matching/MatchingV2Service.php:91`

- **Impact** : un provider qui passe online via `/api/provider/presence-v2/online` reste `is_online=false` et est exclu du dispatch ASAP.
- **Preuve** : deux `ProviderPresenceService` homonymes. Le v2 (`App\Services\Presence`) écrit `provider_presence` mais ne touche jamais `ProviderProfile.is_online` . Or `MatchingV2Service:91` et `AiDispatchService:114` filtrent sur `$profile→is_online` (legacy). Aucun observer/bridge (`grep` = vide). Chaque système a son propre cron de stale.
- **Correctif** : une seule source de vérité de présence pour le dispatch (faire que Presence v2 mette aussi à jour `is_online` , ou faire lire `provider_presence` par le matching) ; supprimer le système non retenu ; lever l'ambiguïté de nommage.

Thème G — Performance

G1. [HIGH] N+1 massif dans la matérialisation des campagnes marketing `app/Services/Marketing/CampaignEngine.php:62-99`

- **Impact** : ~35 000 requêtes pour un segment de 5000 membres × 3 steps, dans une seule transaction longue (verrous, timeout) même en queue.

- **Preuve** : `User::find($uid)` par membre (l.64) ; `optOut→isOptedOut()` = `exists()` par membre×step (OptOutService:64-69) ; `MarketingCampaignRecipient::where('idempotency_key')→first()` par membre×step (l.87-88).
- **Correctif** : `whereIn('id',$memberIds)→get()→keyBy('id')` , précharger opt-outs et idempotency_keys en une requête chacun, insert bulk chunké.

Thème H — Tests (fausse confiance)

H1. [HIGH] La suite entière des AI write actions ne s'exécute JAMAIS en CI `tests/Unit/Assistant/AssistantWriteActionsTest.php:33`

- **Impact** : 100% des tests couvrant `cancel_booking` / `create_booking` / `resolve_dispute` (actions touchant argent et état) sont skippés en permanence ; pipeline vert trompeur.
- **Preuve** : `setUp()` fait `markTestSkipped` si `database.default ≡ 'sqlite'` , or `phpunit.xml` fixe `DB_CONNECTION=sqlite :memory:` (config CI). Combiné au seuil de couverture non-bloquant.
- **Correctif** : job CI MySQL/Postgres dédié (ou rendre le schéma compatible SQLite) ; faire échouer le build si un test critique money/assistant est skippé.

4. Findings medium & low

#	Sév	Dimension	Titre	Fichier:ligne
M1	medium	security:authz	Routes <code>/api/provider/*</code> gardées uniquement par <code>auth:sanctum</code> (aucun gating de rôle)	routes/api/provider.php:30-159
M2	medium	security:authz	Achat d'assurance sans vérif d'ownership du booking	Api/Client/InsuranceController.php:27-65 ; Insurance/InsuranceService.php:33-66
M3	medium	security:injection	Photos personnelles sur disk PUBLIC accessible sans auth	Livewire/Employe/MesRendezVous.php:271
M4	medium	money	Double déduction de commission au ledger wallet (provider sous-crédité, bug figé dans le test)	Payments/ProviderWalletService.php:58-125
M5	medium	money	Insert wallet brut non idempotent dans le cron payouts (double crédit possible)	ProcessProviderPayouts.php:80-124
M6	medium	data	Schéma <code>customer_credits</code> doublement défini (wallet vs crédit-par-booking), modèle désaligné	mig 2026_05_20_030001_align_customer_credits_schema.php:28-58
M7	medium	data	Colonne <code>tenant_id</code> morte sur users + trait BelongsToTenant inexistant (Tenancy v2 supprimé)	mig 2026_05_20_020001_add_tenant_id_to_users_optional.php:29-31
M8	medium	data	<code>bookings.surface</code> (varchar tranches) vs <code>surface_m2</code> (int) — double colonne ambiguë	mig 2026_05_17_120006_fix_bookings_surface_to_string_type.php:43-45
M9	medium	reliability	Webhooks Stripe Connect échouant transitoirement = perdus (tries=1, backoff mort, aucun rescanner FAILED/next_retry_at)	Jobs/Payments/ProcessStripeWebhookJob.php:20
M10	medium	reliability	Payout insert SQL brut sans idempotency_key + SELECT hors transaction (TOCTOU sur invocation manuelle)	ProcessProviderPayouts.php:86
M11	medium	compliance	<code>audit_events</code> stocke l'email en clair dans actor_label/subject_label (hors redaction, hors anonymisation)	Audit/AuditService.php:322
M12	medium	compliance	PII marketing (email + téléphone) loggée en clair via Log::info	Marketing/CampaignEngine.php:259
M13	medium	compliance	Fichiers d'export RGPD (dump complet PII) jamais supprimés du disque après expiration	Gdpr/DataExportService.php:42
M14	medium	compliance	Aucune rétention/purge pour <code>analytics_events</code> (user_id conservé indéfiniment, non anonymisé)	Gdpr/RetentionPolicyService.php:14

#	Sév	Dimension	Titre	Fichier:ligne
M15	medium	mobile	<code>TaskManager.defineTask</code> au top-level d'un module importé par 3 écrans (risque Expo Go SDK 53+)	provider/src/tracking/useBackgroundLocation.ts:1-19
M16	medium	mobile	File offline non fonctionnelle : replay sans baseUrl ni token ; offlineAwareMutation utilisé nulle part	shared/src/lib/offlineQueue.ts:37-41
M17	medium	mobile	État de session non réinitialisé après échec de refresh (UI authentifiée fantôme)	shared/src/api/client.ts:73-75
M18	medium	archi	Overrides de feature flags admin jamais lus à l'exécution (kill-switch fantôme)	FeatureFlag/FeatureFlagService.php:27
M19	medium	archi	OnboardingV2 (engine+API+tests) entièrement contourné par le wizard web + API legacy	Livewire/Provider/Onboarding/ProviderOnboardingWizard.php:90
M20	medium	perf	<code>bookings.date</code> sans index alors qu'elle est filtrée/triée partout ; <code>whereDate()</code> empêche l'index	mig 2026_05_04_000006_create_booking_tables.php:151,181-187
M21	medium	perf	Dashboard admin : 9+ COUNT séparés sur <code>created_at</code> non indexé, dont 7 en boucle	Livewire/Admin/AdminHomeDashboard.php:47-65
M22	medium	tests	StripePaymentWebhookChainTest ne teste pas la chaîne webhook → processeur (bypass + skip si colonne manque)	tests/Feature/Integration/StripePaymentWebhookChainTest.php:68
M23	medium	tests	Code de vérification mission (QR start/end) sans test direct des chemins d'erreur ; attempts jamais utilisé pour bloquer (brute-force possible)	Services/Missions/MissionVerificationCodeService.php:40
M24	medium	tests	Guard d'autorisation de paiement validé par un <code>catch \Throwable</code> générique (fausse confiance)	tests/Feature/Payments/MissionPaymentServiceTest.php:42
M25	medium	tests	Tests avec foreign keys désactivées (<code>DB_FOREIGN_KEYS=false</code>) + skips silencieux sur schéma manquant	phpunit.xml:34
L1	low	security:authz	Modèle RendezVous (Booking) en <code>\$guarded = []</code> — mass-assignment latent	Models/RendezVous.php:8-12
L2	low	security:injection	Sanitisation XSS par <code>strip_tags</code> allowlist ne retire pas les attributs (non exploitable, ContractRenderer échappe déjà)	livewire/client/client-contract-sign.blade.php:81
L3	low	security:injection	<code>\$guarded</code> vide sur RendezVous + RecurringBookingSeries (defense-in-depth)	Models/RendezVous.php:12 ; RecurringBookingSeries.php:17
L4	low	data	Colonnes type-FK ajoutées en masse par migrations « fix/roundN » sans <code>>constrained()</code>	mig 2026_05_13_232801_fix_runtime_schema_round6.php:13-23
L5	low	data	Idempotence <code>kyc_webhook_events</code> fragilisée par UNIQUE sur colonne nullable (NULL ≠ NULL en MySQL)	mig 2026_05_18_140004_create_kyc_webhook_events_table.php:15-31
L6	low	reliability	Fallback aléatoire <code>external_event_id</code> (Str::random) annule l'idempotence sur rejeu KYC/SMS/Insurance	Webhooks/KycWebhookController.php:52
L7	low	compliance	Données KYC/KYB sensibles (metadata, date_of_birth) stockées sans cast <code>encrypted</code>	mig 2026_05_19_150001_create_kyb_v2_tables.php:111
L8	low	mobile	PaymentCheckout : échec d'init du PaymentSheet avalé (catch vide) → spinner permanent, pas de retry	client/src/screens/PaymentCheckoutScreen.tsx:31-34
L9	low	archi	Méthodes privées mortes (<code>bookingAmount</code> , <code>scheduledAt</code>) dans CancellationFeeCalculator	Services/Cancellation/CancellationFeeCalculator.php:217

#	Sév	Dimension	Titre	Fichier:ligne
L10	low	perf	N+1 en cascade (~6×N) dans le scoring de matching (chemin synchrone de dispatch)	Services/Matching/MatchingScoreEngine.php:69-82,173,195-218
L11	low	perf	<code>ClientDashboard::coverageZoneIds()</code> recalculé 3× par render (non mémoisé)	Livewire/ClientDashboard.php:43-74
L12	low	perf	SubscriptionScheduler : <code>exists()</code> + <code>create</code> par abonnement en boucle, sans index date	Services/Subscription/SubscriptionScheduler.php:13-41

5. Plan d'action priorisé (top 10)

#	Action	Effort	Risque adressé
1	Marquer les missions destination-charge (<code>payout_status='auto_transferred'</code>) et les exclure de la Phase 2 du cron payouts	M	A1 critical — double paiement automatique quotidien
2	Ajouter <code>idempotency_key</code> au <code>Transfer::create()</code> manuel + <code>lockForUpdate</code> sur les bookings sélectionnés	S	A2/M10 — double virement sur crash/concurrence
3	Renommer le cron erasure en <code>gdpr:execute-erasures</code> + test de résolvabilité des commandes schedulées	S	D1 — droit à l'oubli jamais exécuté (art.17)
4	Garde de transition <code>BookingStatus::canCancel()</code> dans CancellationEngine ET CancelBookingService	S	B1 — re-annulation/refund d'un booking complété
5	Policy + <code>authorize()</code> sur tout le module Quality/Inspection (contrôleurs client+provider+service) ; ajouter <code>role:employee</code> au groupe provider	M	C1/M1 — IDOR lecture+écriture, signature au nom du client
6	Implémenter les endpoints <code>qrStart/qrEnd</code> client (lifecycle + capture) et transmettre <code>endcode</code> provider	M	E1/E2 — cœur du flow QR cassé sur mobile
7	Choisir une source de vérité unique pour annulation et présence ; router tous les canaux, supprimer le legacy non retenu	L	F1/F2 — frais/refund divergents, dispatch ASAP cassé
8	Étendre <code>anonymizeUser()</code> (bookings, feedback, complaint_cases, notifications, audit_events, analytics) + test PII résiduelle	M	D2/M11/M14 — PII survivant à l'effacement
9	Faire lire les overrides DB par <code>FeatureFlagService::isEnabled()</code> (cache par requête) + test d'intégration	S	M18 — kill-switches admin inopérants en incident
10	Job CI MySQL/Postgres pour les suites money/assistant/RGPD ; échec si test critique sauté ; <code>DB_FOREIGN_KEYS=true</code>	M	H1/M22/M23/M24/M25 — fausse confiance des tests

Effort : S ≤ 1j, M ≤ 3j, L > 3j ou refonte transverse.

6. Zones vérifiées saines

- **Isolation par utilisateur (cœur du flux)** : bookings, missions, wallet, disputes, invoices, ratings, availability, favorites, devices, insurance cancel/claims vérifient l'ownership via `$request→user()→id` ou des scopes dédiés ; RatingService/MissionLifecycle valident l'éligibilité. (Le module Quality est la seule exception — C1.)
- **Injection & secrets** : dimension globalement saine ; pas de secret en dur, requêtes paramétrées, ContractRenderer échappe toutes les variables via `e()` (la fragilité `strip_tags` L2 n'est pas exploitable).
- **Webhooks entrants** : idempotence par `firstOrCreate` sur `external_event_id`, dispatch async, dead-letter pour Stripe/WebhooksV2 ; webhooks Stripe eux-mêmes idempotents/lockés ; clawback proportionnel correct.
- **Réfuté en vérification — sur-retrait illimité du wallet** : `requestWithdraw` ne déclenche aucun appel Stripe ; `ProcessProviderPayouts` itère des bookings, jamais des `ProviderPayout`. La chaîne d'exploitation vers une perte d'argent est rompue (reste un bug d'intégrité de ledger, low).
- **Réfuté — « split-brain » bookings client/provider FK NULL** : le hook `Booking::booted()` + `HasLegacyBookingAliases::syncLegacyAliases()` (Booking.php:351-355) miroite bidirectionnellement `client_id*customer_user_id` et `employee_id*assigned_provider_user_id` AVANT chaque insert ; les colonnes FK ne sont pas NULL et la contrainte FK s'applique réellement.
- **Réfuté — 4 relations Eloquent alimentées par des chemins divergents** : l'evidence était mal attribuée (`MultiTradeBundleService:121` modifie un `MultiTradeBundleItem`, pas un Booking ; la vraie création écrit `employee_id` comme `CreateBookingAction`) ; la relation `provider()` citée n'existe pas. Aucun NULL silencieux.

- **Tests money de qualité** : `RefundClawbackTest` (clawback proportionnel, idempotence webhook/refund, double-débit, edge cases avec assertions monétaires) et `StripeWebhookControllerTest` (signature/replay/secret manquant) sont solides.
- **Dashboards client/employé** : correctement eager-loadés, computed properties cachées (à l'exception des points perf signalés sur l'admin et le matching).
- **Mobile — infrastructure** : SecureStore pour le token, imports dynamiques d'expo-notifications, interceptor 401+refresh, ErrorBoundary, aucun secret en AsyncStorage.
- **Trait `BelongsToTenant`** : confirmé supprimé du code (seul subsiste la colonne morte `tenant_id` — M7).
- **Crons** : `withoutOverlapping` en place ; dispatch/escalade correctement conçus.